Desarrollo Multiplataforma con React Native

Día 1: Introducción, Setup y Primeras Apps

Carlos Rodríguez Contreras
carlos.mailhub@gmail.com
karrocon

20 de abril de 2026

Construyamos una app móvil real

Agenda del día

- Qué vamos a construir
- ¿Qué es React Native?
- Cómo funciona
- Setup con Expo
- Primer código
- Estilos
- Ejercicios

Objetivo del día

Que **todos** ejecuten una app en su máquina.
Si lo conseguimos → ✔ éxito.

Horario orientativo

16:00 Intro + qué construimos
16:30 **PARADA: instalación**
17:30 React Native + cómo funciona
18:00 **DESCANSO 15 min**
18:15 Primer código + estilos
19:30 **PARADA: ejercicios**
20:30 Estado + cierre
21:00 Fin

¿Qué vamos a construir?

Primero lo primero: el destino

Lo que construiremos: Pokedex App

Al final del curso tendrás:

- ✔ App móvil que funciona en Android / iOS
- ✔ Lista de Pokémon desde API real
- ✔ Pantalla de detalle con estadísticas
- ✔ Navegación entre pantallas
- ✔ Diseño propio

Tecnología:
JavaScript + React Native + PokéAPI

¿Por qué Pokémon?

- API gratuita
- Datos ricos
- Visualmente atractivo
- Todo el mundo la conoce

El camino: 8 días

Día	Tema	Resultado
1	Intro + Setup	App corriendo en tu máquina
2	Fundamentos React	Contador + formulario
3	Navegación	App con 3 pantallas
4	Consumo de API	Lista de datos remotos
5–6	Pokedex (parte 1 y 2)	App funcional
7–8	Mejoras + cierre	App terminada y publicada

➔ **Hoy: Día 1.** Objetivo mínimo: app corriendo.

Antes de instalar — Cuentas necesarias

GitHub

Sube tu código y colabora.

Crear cuenta:

`github.com/signup`

≈ 2 minutos

Expo

Publica y comparte tu app.

Crear cuenta:

`expo.dev/signup`

≈ 2 minutos

Expo Go

Prueba la app en tu móvil.

Instalar:
App Store o Google Play
Busca «Expo Go»

≈ 1 minuto

ℹ Si ya tienes cuenta en GitHub o Expo, puedes saltarte ese paso.

Instalación del entorno — Paso a paso

Opción A — Script automático (recomendado):

```
# PowerShell como Administrador
Set-ExecutionPolicy -Scope Process -
    ExecutionPolicy Bypass
.\instalar-entorno.ps1
```

El script instala y configura:

- ✔ Node.js LTS 20
- ✔ Git for Windows
- ✔ Visual Studio Code + extensiones
- ✔ Expo CLI (`expo login`)
- ✔ Clona el repositorio del taller

⚠ Si el script falla

Opción B — Manual:

1. `nodejs.org` → descargar LTS
2. `code.visualstudio.com`
3. `git-scm.com`
4. Expo Go en el móvil

💻 Si algo no va, no te preocupes: el profesor te ayudará.

Clonar el repositorio del taller

Opción A — Con Git:

```
git clone https://github.com/karrocon/
  taller-react-native-fp.git
cd taller-react-native-fp
```

Opción B — Descargar ZIP:

1. Entra en `github.com/karrocon/taller-react-native-fp`
2. Pulsa **Code** → **Download ZIP**
3. Descomprime en tu carpeta de trabajo

En el repo encontrarás

- Slides de cada día (PDF)
- Ejercicios con soluciones
- Scripts de instalación
- Recursos y referencias

➔ El script `instalar-entorno.ps1` ya clona el repo automáticamente.

Parada: Instalación

16:30 — 17:30 Instalación guiada

Mientras instalan, el profesor explica:

- Qué estamos instalando y por qué
- Arquitectura de un proyecto Expo
- Dudas del bloque anterior

Guía: `guia-setup-alumno.md`

Checklist mínimo

- [ ] Cuenta GitHub creada
- [ ] Cuenta Expo creada
- [ ] Expo Go en el móvil
- [ ] Node 18+ instalado
- [ ] VS Code abierto
- [ ] Repo clonado

✔ Si todos llegaron aquí → continuar

¿Qué es React Native?

Y por qué es diferente a lo que conoces

El problema que resuelve

Sin React Native:

- ✖ Android → Java / Kotlin
- ✖ iOS → Swift / Obj-C
- ✖ Dos equipos, dos codebases
- ✖ El doble de trabajo, el doble de bugs

Con React Native:

- ✔ **Un solo código**
- ✔ JavaScript + React (ya lo conoces)
- ✔ Compila a componentes **nativos reales**
- ✔ Mantenido por **Meta** (Facebook)

React Native **no es una web** envuelta en una app.
Genera componentes nativos reales.

Comparativa de tecnologías

Tecnología	Lenguaje	Plataforma	Rendimiento
HTML / CSS / JS	JavaScript	Web	Alto (web)
Android nativo	Kotlin	Solo Android	Máximo
iOS nativo	Swift	Solo iOS	Máximo
React Native	JavaScript	Android + iOS	**Muy alto**
Flutter	Dart	Android + iOS	Muy alto

👍 Apps que usan React Native: **Facebook, Instagram, Shopify, Discord, Skype**

¿Cómo funciona por dentro?

(Lo suficiente para no perderse)

El puente (Bridge)

Tú escribes:

```
<Text>Hola mundo</Text>
```

El dispositivo muestra:

- `TextView` (Android)
- `UILabel` (iOS)

Componentes web vs React Native

Web (HTML)		React Native
`<div>`	→	`<View>`
`<p>`, `<span>`	→	`<Text>`
`<img>`	→	`<Image>`
`<input>`	→	`<TextInput>`
`<button>`	→	`<Button>` / `<TouchableOpacity>`
`<ul>`/`<li>`	→	`<FlatList>`

Importante

En React Native **todo el texto** debe estar dentro de `<Text>`.
Si pones texto suelto dentro de `<View>` → error.

Descanso — 15 minutos

18:00 — 18:15 Descanso

15 minutos

Al volver: Expo y primer proyecto

Setup con Expo

La forma más rápida de empezar

¿Por qué Expo?

Sin Expo (React Native CLI):

- ✖ Necesitas Android Studio instalado
- ✖ Necesitas Xcode (solo en Mac)
- ✖ Configuración compleja
- ✖ Horas perdidas el primer día

Con Expo:

- ✔ Solo Node.js
- ✔ Un comando para crear el proyecto
- ✔ Preview instantánea en móvil (QR)
- ✔ También funciona en navegador

Consejo

Para producción real puedes migrar a React Native CLI. Para aprender, Expo es perfecto.

Paso a paso: crear tu primera app

1. Verificar Node (necesitas 18+):

```
node --version
```

2. Crear proyecto:

```
npx create-expo-app miApp
cd miApp
```

3. Arrancar:

```
npm start
```

ℹ En clase trabajamos sobre el repo clonado.
Este comando es para proyectos propios futuros.

Ejecutar en móvil

1. Instala **Expo Go** en tu móvil
2. Escanea el código QR
3. Misma red WiFi

Alternativa web

Pulsa `w` en el terminal para abrir en navegador

Estructura del proyecto creado

```
miApp/
   App.js                <- aqui empezamos
   package.json
   app.json                 <- config Expo
   assets/            <- imagenes, fuentes
   node_modules/              <- no tocar
```

Lo importante ahora:

- Solo necesitas abrir `App.js`
- Todo lo demás viene configurado
- Los cambios se ven al guardar (hot reload)

Hot reload: cada vez que guardas, la app se actualiza automáticamente.

El App.js inicial

```
import { StatusBar }
  from 'expo-status-bar';
import { StyleSheet, Text, View }
  from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js!</Text>
      <StatusBar style="auto" />
    </View>
  );
}
```

```
const styles =
  StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

➜ Cambia el texto y guarda. Verás el cambio al instante.

Primer código

JSX, componentes y estructura básica

JSX — HTML dentro de JavaScript

```
// Esto es JSX
export default function App() {
  return (
    <View>
      <Text>Hola mundo</Text>
    </View>
  );
}
```

Reglas JSX:

- Siempre un elemento raíz
- Etiquetas siempre cerradas: `<View />` o `</View>`
- Clases → no existen en RN
- JS dentro de `{ }`

JSX **no es HTML**. Es JavaScript disfrazado. El compilador lo convierte en llamadas a funciones.

Variables y expresiones en JSX

```
export default function App() {
  const nombre = 'Carlos';
  const edad = 25;

  return (
    <View style={{ padding: 20 }}>
      <Text>Nombre: {nombre}</Text>
      <Text>Edad: {edad}</Text>
      <Text>El doble: {edad * 2}</Text>
      <Text>{'Mayusculas: ' + nombre.toUpperCase()}</Text>
    </View>
  );
}
```

Dentro de { } puedes poner **cualquier expresión JavaScript** que devuelva un valor.

Componentes básicos (1/2) — Estructura

```
import { View, Text, Image }
  from 'react-native';

export default function App() {
  return (
    <View>
      <Text>Hola</Text>
      <Image
        source={{ uri: 'https://...' }}
        style={{ width:100, height:100 }}
      />
    </View>
  );
}
```

View	Contenedor (= `div`)
Text	Todo el texto
Image	Local o URL

Todo texto **debe** ir dentro de `<Text>`.

Componentes básicos (2/2) — Interacción

```
import { TextInput, Button,
         TouchableOpacity }
  from 'react-native';

<TextInput
  placeholder="Escribe..."
  onChangeText={setText}
/>

<Button
  title="Pulsa"
  onPress={() => alert('ok')}
/>

<TouchableOpacity onPress={fn}>
  <Text>Boton custom</Text>
</TouchableOpacity>
```

TextInput	Input texto
Button	Botón nativo
Touchable	Botón custom
FlatList	Listas

Usa **TouchableOpacity** para botones con diseño propio.

Estilos en React Native

No es CSS... pero se parece

Estilos — Inline vs StyleSheet

Inline:

```
<View style={{ padding: 20 }}>
  <Text style={{ fontSize: 24,
                 color: 'blue' }}>
    Hola
  </Text>
</View>
```

StyleSheet (recomendado):

```
const styles = StyleSheet.create({
  container: { padding: 20 },
  titulo: {
    fontSize: 24,
    fontWeight: 'bold',
  },
});
<Text style={styles.titulo}>
```

Diferencias vs CSS:

- **camelCase**:
  `font-size` → `fontSize`
- Sin px: `fontSize: 16`
- Flexbox por defecto
- Sin herencia
- Sin selectores

Flexbox igual que CSS
pero `flexDirection: 'column'`
por defecto.

Flexbox básico

```
// Centrar todo
centrado: {
  flex: 1,
  justifyContent: 'center',
  alignItems: 'center',
},
// Fila horizontal
fila: {
  flexDirection: 'row',
  justifyContent: 'space-between',
},
```

flex: 1	todo el espacio
justifyContent	eje principal
alignItems	eje secundario
row	horizontal
column	vertical (def.)

Truco: `flex: 1` en el contenedor raíz = pantalla completa.

¡A practicar!

3 ejercicios — de menos a más

Ejercicio 1 — Tarjeta de presentación

Enunciado

Crea una app que muestre tu nombre y edad en pantalla.
Bonus: añade también tu ciudad y un emoji favorito.

Solución base:

```
import { Text, View } from 'react-native';

export default function App() {
  return (
    <View style={{ padding: 20 }}>
      <Text>Nombre: Juan</Text>
      <Text>Edad: 20</Text>
    </View>
  );
}
```

Ejercicio 2 — Texto con estilo

Enunciado

Añade estilos a tu tarjeta:

- ✔ Nombre grande y negrita
- ✔ Fondo de color suave
- ✔ Texto centrado

💡 Usa ´flex: 1+ ´justifyContent+ ´alignItemsén el container.

Pista:

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#e8f4fd',
  },
  nombre: {
    fontSize: 28,
    fontWeight: 'bold',
    color: '#1a73e8',
  },
});
```

Ejercicio 3 — Tarjeta con imagen

Enunciado

Añade una imagen a tu tarjeta. Puedes usar cualquier URL pública.
Bonus: añade un borde redondeado a la imagen.

```
<Image
  source={{ uri: 'https://picsum.photos/200' }}
  style={{
    width: 120,
    height: 120,
    borderRadius: 60,
    marginBottom: 16,
  }}
/>
```

`borderRadius: 60` (mitad del ancho/alto) = imagen circular

Parada: Ejercicios en clase

20:00 — 20:30 Trabajo autónomo guiado

Los alumnos trabajan en los 3 ejercicios.

El profesor:

- Rota por los puestos
- Ayuda con errores comunes
- Muestra solución en proyector si hay bloqueo general

Soluciones: `ejercicios-dia01.md`

Criterio de éxito

- App con nombre y edad
- Estilos aplicados
- Imagen circular visible

Bonus: layout en dos columnas

Estado y eventos

Intro a useState — preview del Día 2

useState — Tu primera pieza de estado

```
import { useState } from 'react';
import { View, Text, Button }
  from 'react-native';

export default function App() {
  const [contador, setContador]
    = useState(0);

  return (
    <View style={{ padding: 40 }}>
      <Text>Contador: {contador}</Text>
      <Button
        title="+1"
        onPress={() =>
          setContador(contador + 1)}
      />
    </View>
  );
}
```

Tres conceptos:

1. `useState(0)` → valor inicial
2. `contador` → leer
3. `setContador()` → escribir

Nunca modifiques el estado directamente:
`contador = 1` ✖
Usa siempre `set...()`

Ejercicio bonus — Contador interactivo

Ejercicio extra

Añade un botón **+1** y un botón **Reset** a tu tarjeta.
Muestra el contador en pantalla.

```
export default function App() {
  const [n, setN] = useState(0);
  return (
    <View style={{ padding: 30 }}>
      <Text style={{ fontSize: 32 }}>{n}</Text>
      <Button title="+1"
        onPress={() => setN(n + 1)} />
      <Button title="Reset"
        onPress={() => setN(0)} />
    </View>
  );
}
```

Problemas comunes y soluciones

Problema	**Solución**
Expo no arranca	Reinstalar Node 18+ / limpiar: `npm cache clean --force`
No conecta en móvil	Usar la opción web (`w`) o verificar que estás en la misma WiFi
Error de dependencias	Borrar `node_modules` y ejecutar `npm install`
Pantalla en blanco	Revisar consola — casi siempre hay un error de sintaxis
`Text strings...`	Hay texto fuera de `<Text>` — revisar el JSX

Troubleshooting

Los errores más frecuentes del Día 1

Errores más frecuentes

Error	Causa y solución
`expo: command not found`	No está instalado. Ejecuta: `npm install -g expo-cli`
`node: command not found`	Node no instalado o no en PATH. Reinstalar desde nodejs.org
Pantalla en blanco	Error en el código. Abre la consola **(j)** y busca el mensaje rojo
QR no conecta	Revisar que móvil y PC están en la **misma red WiFi**
`Text strings must be...`	Hay texto fuera de `<Text>` en el JSX

Error: cannot find module

✖ Error típico

```
Error: Cannot find module 'react-native'
```

Causa: `node_modules` no existe o está corrupto.

Solución:

1. Cierra el servidor (Ctrl+C)
2. Borra `node_modules/`
3. Ejecuta `npm install`
4. Vuelve a ejecutar `npm start`

```
rm -rf node_modules
npm install
npm start
```

En Windows borra la carpeta desde el Explorador o usa PowerShell.

Error: Text strings must be within <Text>

✖ **Incorrecto:**

```
<View>
  Hola mundo
</View>
```

✔ **Correcto:**

```
<View>
  <Text>Hola mundo</Text>
</View>
```

⚠ Regla de oro

Todo texto visible debe estar dentro de <Text>.

Incluye espacios y saltos de línea sueltos dentro de <View>.

Móvil no conecta / Error de sintaxis JSX

QR no conecta:

- ☐ Misma red WiFi (PC y móvil)
- ☐ Firewall no bloquea puerto 8081
- ☐ Expo Go actualizado
- ☐ Prueba `npx expo start --tunnel`

Plan B: pulsa `w` → abrir en navegador

</> JSX con dos raíces:

```
// ERROR
return (
  <View/><View/>
);
// OK: usa Fragment
return (
  <>
    <View/>
    <View/>
  </>
);
```

Resumen del día

Hoy hemos aprendido:

- ✔ Qué es React Native y para qué sirve
- ✔ Cómo funciona el bridge JS ↔ nativo
- ✔ Crear un proyecto con Expo
- ✔ Componentes: `View`, `Text`, `Image`
- ✔ JSX y expresiones `{ }`
- ✔ Estilos con `StyleSheet`
- ✔ Flexbox básico
- ✔ Primer vistazo a `useState`

Día 2 — Próxima sesión

- **useState** — estado
- Eventos (botones)
- Inputs controlados
- Renderizado condicional

➜ Construiremos un contador y un formulario

¿Preguntas?

Antes de salir:

✔ La app funciona en tu máquina
✔ Sabes dónde está el `App.js`
✔ Entiendes qué es JSX

Si no es así → **ahora es el momento de preguntar**